

编译原理 第二次作业

姓名：朱首赫

学号：2023K8009906029

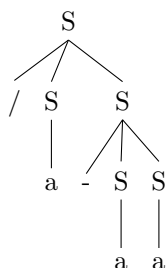
1 练习 2.2.1

考虑下面的上下文无关文法：

$S \rightarrow -SS \mid /SS \mid a$

- 1) 试说明如何使用文法生成串/a-aa
- 2) 试为这个串构造一颗语法分析树
- 3) 该文法生成的语言是什么？
- 4) 该文法具有二义性吗？为什么？

- 1) 采用最左推导： $\underline{S} \Rightarrow / \underline{S} S \Rightarrow / a \underline{S} \Rightarrow / a - \underline{S} S \Rightarrow / a - a \underline{S} \Rightarrow / a - aa$
- 2) 最左推导的语法分析树如下：



- 3) 该文法生成的语言是由运算符 ‘/’、‘-’，和操作数 ‘a’ 组成的所有前缀表达式的集合。
- 4) 该文法不具有二义性。对于任何一个前缀表达式，总可以从左向右扫描依次得到每部操作的运算符和对应的操作数，即运算语义是唯一的。

2 练习 2.1.2

考虑文法： $num \rightarrow 0 \mid 101 \mid num0 \mid num \ num$

- 1) 证明：用该文法生成的所有二进制串的值都能被 5 整除（提示：对语法分析树的结点数目，即推导步数，使用数学归纳法）
- 2) 上面的文法是否能够生成所有能被 5 整除的二进制串？

- 1) 证明：推导步数为 1 时：二进制串 0, 101 显然可以被 5 整除。推导步数为 2 时：第一步只能是 $num \rightarrow num0$ ，那么第二步得到的终结符串为 00 或 1010，显然可以被 5 整除。

假设推导步数小于等于 n 时，得到的二进制串都能被 5 整除。

推导步数为 $n+1$ 时，第一步推导要么为 $num_0 \rightarrow num_1 0$ ，要么为 $num_0 \rightarrow num_1 num_2$ ，由假设得 num_1 、 num_2 最终得到的二进制串均可被 5 整除。对于第一种情况， $num_0.v = num_1.v \times 2$ ， $num_0.v$ 可以被 5 整除；对于第二种情况，设 num_2 最终得到的二进制串位数为 a ，则 $num_0.v = num_1.v \times 2^a + num_2.v$ ，可以被 5 整除。

因此，由数学归纳法可得，该文法生成的所有二进制串的值都能被 5 整除。

- 2) 该文法不能够生成所有能被 5 整除的二进制串，如 1111 就不能被生成，因为 num 只能生成 101 通过移位和拼接得到的字符串。

3 练习 2.1.3

构建一个语法制导翻译方案，该方案把算术表达式从中缀表示方式翻译为运算符在运算分量之后的后缀表示方式。例如， $xy-$ 是表达式 $x-y$ 的后缀表示。给出输入 $8-3+4$ 和 $8-3*4$ 的注释分析树。p.s. 注意区分翻译语法分析树和注释语法分析树的概念（编译原理中文第 2 版 Page.196）

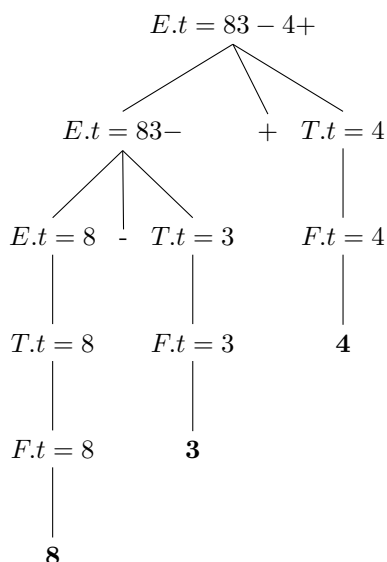


图 1: $8-3+4$ 的注释分析树

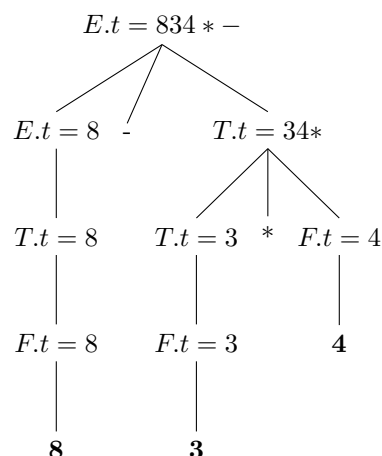


图 2: $8-3*4$ 的注释分析树

4 练习 2.1.4

从 C99 标准开始，C 语言的 for 语句中初始化子句可以是声明，例如：

```
for ( int i = 0; i < n; i++ )
```

在 C++ 中允许类似的语法，但区别是：C++ 中初始化语句的作用域与循环语句的作用域一致，而在 C 中循环语句的作用域嵌套于初始化语句的作用域中。例如

```
for (int i = 0; ; ) long i = 1; // 在 C 中合法，在 C++ 中非法
```

参考龙书图 2-37 中的实现，试讨论在两种 for 语句实现中分别应该如何实现符号表来正确区分变量和避免重复声明。

对于 C 语言，当语法分析器扫描到 for 关键字时，执行 $\text{top} = \text{new Env}(\text{top})$ 创建一个新的符号表，然后完成初始化子句扫描后，当遇到循环体左括号 $\{$ 时，再次执行 $\text{top} = \text{new Env}(\text{top})$ 创建一个新的符号表。

对于 C++，当语法分析器扫描到 for 关键字时，同样执行 $\text{top} = \text{new Env}(\text{top})$ 创建一个新的符号表，但是当遇到循环体左括号 $\{$ 时不再创建新的符号表。此后遇到变量声明时，在 put 之前先尝试 get，若返回值不为 NULL，则遇到重复声明，直接报错。

5 练习 2.1.5

给出一个文法，生成所有被 3 整除的非负整数的二进制串，并给出证明（要求不能以 0 开头，除非该数本身为 0）

文法 $G[S]$ 生成所有被 3 整除的非负整数的二进制串：

$$S \rightarrow 0 \mid 1A$$

$$A \rightarrow 0B \mid 1C$$

$$B \rightarrow 0A \mid 1B$$

$$C \rightarrow 0C \mid 1A \mid \epsilon$$

证明：首先， $S \rightarrow 0 \mid 1A$ 保证了能够生成 0 和非 0 数必定以 1 开头的要求。然后，需要证明从 $S \rightarrow 1A$ 开始推导得出的串被 3 整除。

先用数学归纳法证明 A 左侧生成的子串 w_A 除 3 都余 1，B 左侧生成的子串 w_B 除 3 都余 2，C 左侧生成的子串 w_C 都被 3 整除：

1 步推导时，显然 $1A$ 中 $1 \% 3 = 1$ ，符合结论。2 步推导时，可以得到 $10B$ 或 $11C$ ，显然成立。3 步推导时，由 $10B$ 可以得到 $100A$ 或 $101B$ ，由 $11C$ 可以得到 $110C$ 或 $111A$ 或 11 ，显然成立。

假设 n 步推导时结论仍然成立，对于第 $n+1$ 次推导：

(1) 若是 $w_A A \rightarrow w_A 0B \mid w_A 1C$ ，由于 w_A 除 3 余 1，则 $w_B = w_A 0 = 2 \times w_A$ 除 3 余 2， $w_C = w_A 1 = 2 \times w_A + 1$ 被 3 整除，仍然成立；

(2) 若是 $w_B B \rightarrow w_B 0A \mid w_B 1B$ ，由于 w_B 除 3 余 2，则 $w_A = w_B 0 = 2 \times w_B$ 除 3 余 1， $w'_B = w_B 1 = 2 \times w_B + 1$ 除 3 余 2，仍然成立；

(3) 若是 $w_C C \rightarrow w_C 0C \mid w_C 1A \mid w_C$ ，由于 w_C 被 3 整除，则 $w'_C = w_C 0 = 2 \times w_C$ 被 3 整除， $w_A = w_C 1 = 2 \times w_C + 1$ 除 3 余 1， $w'_C = w_C$ 被 3 整除，仍然成立；

因此，上述结论成立。又由于除了 $S \rightarrow 0$ 和 $C \rightarrow \epsilon$ 之外，产生式的右部都有非终结符，所以最后一步推导只能是二者之一，也就是最终得到的二进制串只能是 0 或 w_C ，即可以被 3 整除。